

Rapport de rétroconception du code SAE 4.01



STAGE

Groupe 12

Ali KUS
Nicolas TISGRIS
NYABI Rayan
Anas KIOUAZ
Redon BRAHIMI
Abdessalam KHATTRI

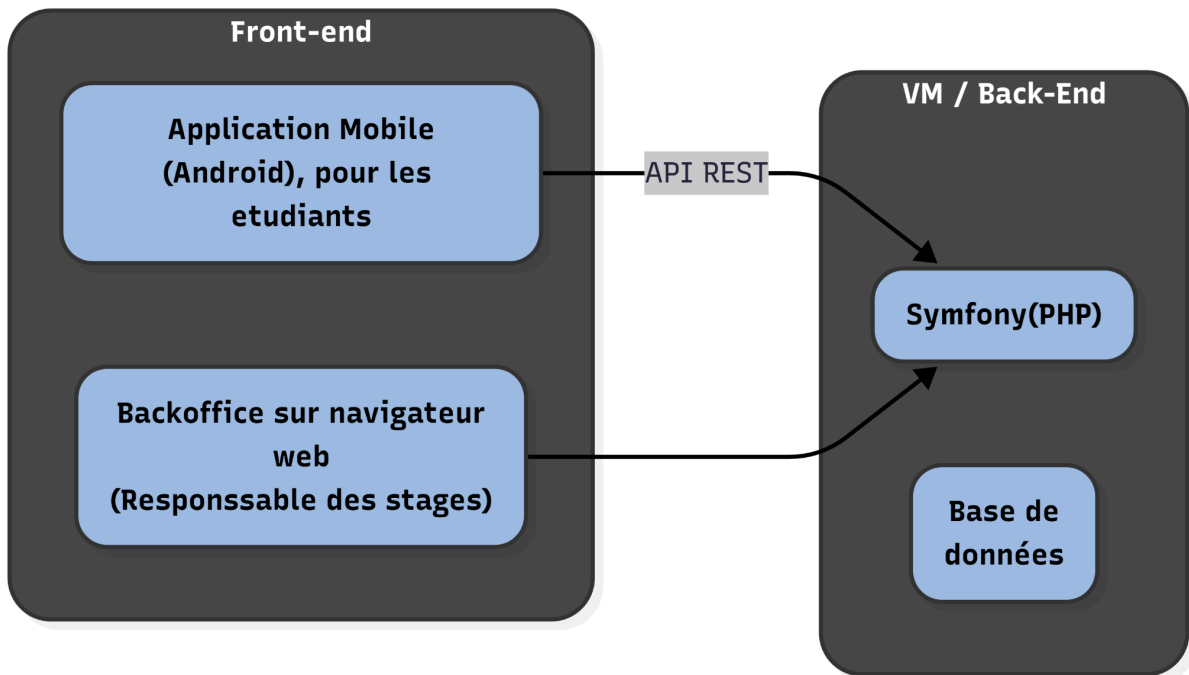
Table des matières

Rapport de rétroconception du code.....	1
SAE 4.01.....	1
Table des matières.....	2
Introduction.....	3
Schéma de Référence.....	3
Analyse de l'organisation du code.....	4
Côté serveur (Symfony).....	4
Côté client (Android - Gradle).....	4
Identification des principales classes et fonctionnalités.....	4
Côté serveur (Symfony - PHP).....	4
Côté client (Android - Java).....	5
Importation et dépendances.....	7
Côté Serveur (Symfony - PHP).....	7
Côté Client (Android - Java).....	7
Liaison entre le serveur et la BD.....	8
Connexion entre le Client et le Serveur.....	9
Exemple de liaison.....	10
Annexes table des matières.....	11

Introduction

Notre mission consistait à réaliser une première analyse de la structure de l'application et des liens entre ses différents éléments, sans entrer dans l'examen détaillé du code. Il s'agissait d'identifier ses principales fonctionnalités et de comprendre comment les fichiers dépendent les uns des autres. Plus largement, l'objectif était de saisir le fonctionnement global de l'application à travers ses différents modules.

Schéma de Référence



Analyse de l'organisation du code

Côté serveur (Symfony)

Principaux dossiers :

src/ → Regroupe l'ensemble du code source de l'application, notamment les contrôleurs, les entités et la logique métier.

config/ → Rassemble les fichiers de configuration nécessaires au fonctionnement de l'application (routes, services, paramètres, etc.).

public/ → Contient les fichiers accessibles depuis l'extérieur, comme le point d'entrée principal (index.php) pour le backoffice.

tests/ → Inclut les tests unitaires et fonctionnels permettant de vérifier le bon fonctionnement du code.

templates/ → Gère les vues de l'application à l'aide du moteur de templates Twig.

Côté client (Android - Gradle)

Principaux dossiers :

app/src/main/ → Contient la structure principale du code de l'application Android.

app/src/main/java/ → Regroupe les classes écrites en Java constituant la logique applicative.

app/src/main/res/ → Stocke les ressources de l'application telles que les fichiers XML de mise en page, les images et les chaînes de caractères.

app/src/test/ → Contient les tests unitaires liés à la partie Android.

.idea/ et .gradle/ → Répertoires liés à la configuration de l'environnement de développement et à la gestion des dépendances avec Gradle.

app/src/main/java/api → Dossier dédié aux classes responsables de la gestion des appels API et des communications avec le serveur.

Identification des principales classes et fonctionnalités

Côté serveur (Symfony - PHP)

Les fichiers montrent que l'application est conçue pour gérer des offres de stage ainsi que les candidatures associées. Les principales classes sont les suivantes :

- **SecurityController.php** : Assure la gestion de l'authentification et des aspects liés à la sécurité.
- **DefaultController.php** : Sert de contrôleur principal pour les actions générales de l'application.
- **Admin/EtatCandidatureController.php** : Permet de gérer les différents états possibles d'une candidature.
- **Admin/OffreController.php** : S'occupe de l'administration des offres de stage.
- **Admin/CandidatureController.php** : Traite la gestion des candidatures des étudiants.
- **Admin/CompteEtudiantController.php** : Gère les comptes des étudiants.
- **Admin/TableauDeBordController.php** : Affiche les informations et statistiques du tableau de bord administratif.

Fonctionnalités principales :

- Authentification des utilisateurs, qu'il s'agisse des étudiants ou des administrateurs.
- Gestion complète des offres de stage : création, modification et consultation.
- Gestion des candidatures : dépôt, validation et suivi de leur évolution.
- Administration des comptes étudiants.
- Mise à disposition d'un tableau de bord destiné à l'équipe administrative.

Côté client (Android - Java)

Les fichiers révèlent que la partie Android assure l'ensemble de l'interface utilisateur, la navigation entre les écrans, l'affichage des données reçues depuis l'API et la gestion des actions de l'utilisateur. Les principales classes sont :

- **LoginActivity.java** — Gère le formulaire de connexion, la récupération des identifiants saisis et l'envoi de la requête d'authentification au serveur.
- **MainActivity.java** — Sert de point d'entrée après connexion, centralise la navigation vers les différentes fonctionnalités.
- **OffreActivity.java** — Affiche le détail complet d'une offre (titre, description, entreprise, dates, etc.) et permet d'interagir avec celle-ci.

- **ListOffresActivity.java** — Récupère et affiche la liste des offres disponibles via l'API, avec gestion de l'adaptateur et du recyclerview.
- **CandidatureActivity.java** — Présente les informations d'une candidature spécifique (état, date, offre associée).
- **ListCandidaturesActivity.java** — Liste toutes les candidatures envoyées par l'utilisateur connecté, avec mise à jour dynamique.
- **CandidatureEditActivity.java** — Permet la modification d'une candidature existante (changement d'état, ajout d'informations, etc.).
- **api/EtatCandidatureEnum.java** — Définit les différents états possibles d'une candidature (ex. : EN_ATTENTE, VALIDÉE, REFUSÉE).

Fonctionnalités principales

- Connexion de l'utilisateur via *LoginActivity*, avec vérification des identifiants et redirection vers l'interface principale.
- Consultation des offres de stage : récupération depuis l'API, affichage sous forme de liste, puis visualisation détaillée dans *OffreActivity*.
- Gestion complète des candidatures : création, consultation, modification et mise à jour via *CandidatureActivity* et *CandidatureEditActivity*.
- Accès à l'historique des candidatures envoyées grâce à *ListCandidaturesActivity*, synchronisée avec les données du serveur.

Affichage principal

Chaque activité possède un fichier XML dédié décrivant précisément sa structure visuelle :

- **activity_login.xml** — Interface du formulaire de connexion, liée à *LoginActivity.java*.
- **activity_main.xml** — Interface principale après connexion, liée à *MainActivity.java*.
- **activity_offre.xml** — Mise en page du détail d'une offre, liée à *OffreActivity.java*.
- **activity_list_offres.xml** — Structure de la liste des offres, liée à *ListOffresActivity.java*.
- **activity_candidature.xml** — Interface d'affichage d'une candidature, liée à *CandidatureActivity.java*.
- **activity_list_candidatures.xml** — Structure de la liste des candidatures, liée à *ListCandidaturesActivity.java*.
- **activity_candidature_edit.xml** — Interface de modification d'une candidature, liée à *CandidatureEditActivity.java*.

Importation et dépendances

Côté Serveur (Symfony - PHP)

Exemple de dépendances entre fichiers

SecurityController.php

- *Importe* : Symfony\Bundle\FrameworkBundle\Controller\AbstractController
Rôle : utiliser les mécanismes internes de Symfony pour gérer l'authentification et la sécurité.
- *Appelle* : les services d'authentification tels que UserRepository et Security pour vérifier les identifiants et gérer les sessions.

OffreController.php

- *Importe* : OffreRepository.php, EntityManagerInterface
Rôle : interroger la base de données afin de récupérer, créer ou modifier des offres de stage.
- *Appelle* : l'entité Offre, qui représente une offre de stage et sert de modèle pour les opérations CRUD.

CandidatureController.php

- *Importe* : CandidatureRepository.php, EtatCandidatureEnum.php
Rôle : gérer l'ensemble du cycle de vie des candidatures (création, mise à jour, validation).
- *Appelle* : EtatCandidatureEnum.php pour déterminer et modifier l'état d'avancement d'une candidature.

TableauDeBordController.php

- *Importe* : OffreRepository.php, CandidatureRepository.php
Rôle : compiler et afficher des statistiques concernant les offres et les candidatures.
- *Appelle* : les repositories pour récupérer les données nécessaires à la génération des indicateurs.

Côté Client (Android - Java)

L'application mobile adopte une architecture MVC, où les activités jouent le rôle de contrôleurs et les adapters assurent la mise en forme et l'affichage des données.

Exemple de dépendances entre fichiers

LoginActivity.java

- *Importe* : UserApiService.java
Utilisé pour : envoyer les identifiants à l'API REST et vérifier l'authentification de l'utilisateur.
- *Appelle* : MainActivity.java après une connexion réussie pour accéder à l'interface principale.

ListOffresActivity.java

- *Importe* : OffreAdapter.java, OffreApiService.java
Utilisé pour : récupérer la liste des offres de stage via l'API et l'afficher grâce à l'adapter.
- *Appelle* : OffreActivity.java lorsqu'un utilisateur sélectionne une offre pour en consulter les détails.

CandidatureActivity.java

- *Importe* : CandidatureApiService.java, EtatCandidatureEnum.java
Utilisé pour : afficher les informations d'une candidature et permettre sa mise à jour.
- *Appelle* : CandidatureEditActivity.java lorsque l'utilisateur choisit de modifier la candidature.

CandidatureEditActivity.java

- *Importe* : CandidatureApiService.java
Utilisé pour : modifier une candidature existante et envoyer les changements à l'API.
- *Appelle* : CandidatureActivity.java après une modification validée afin d'afficher la version mise à jour.

Liaison entre le serveur et la BD

La structure de la base de données repose sur trois niveaux distincts, chacun correspondant à un rôle précis dans le fonctionnement de l'application :

Définition du modèle de données

Les fichiers situés dans **src/Entity/** décrivent la structure des tables. Chaque entité correspond à une table et définit ses champs, ses types, ainsi que les relations (OneToMany, ManyToOne, etc.). Ces fichiers constituent donc le *modèle* de la base.

Exécution des requêtes et accès aux données

Les fichiers du dossier **src/Repository/** contiennent la logique permettant d'interagir avec la base. Ils exécutent les requêtes SQL générées par Doctrine (sélections, filtres, recherches personnalisées) et servent d'interface entre les entités et le reste de l'application.

Gestion des évolutions de la base

Les fichiers du dossier **migrations/** enregistrent toutes les modifications apportées au schéma de la base de données. Ils permettent de créer, mettre à jour ou supprimer des tables et colonnes au fil de l'évolution du projet, en assurant une traçabilité complète des changements.

Connexion entre le Client et le Serveur

L'application mobile repose sur une communication directe avec le serveur Symfony grâce à plusieurs services dédiés, chacun chargé d'un type de données spécifique. Ces services encapsulent les requêtes HTTP et assurent le lien entre les activités Android et l'API du backend.

Interaction mobile ↔ serveur Symfony

Les fichiers **UserApiService.java**, **OffreApiService.java** et **CandidatureApiService.java** regroupent les appels HTTP nécessaires pour :

- envoyer les identifiants de connexion,
- récupérer les offres de stage,
- créer, modifier ou consulter des candidatures.

Ils constituent la couche "Modèle" du côté Android, en fournissant aux activités les données issues du serveur.

Sécurisation côté Symfony

Le fichier **packages/security.yaml** définit les règles d'accès aux différentes routes du backend. Il permet notamment :

- de restreindre certaines URL aux utilisateurs authentifiés,
- de réserver des sections entières de l'application aux comptes **ADMIN**,
- de contrôler les rôles nécessaires pour accéder aux fonctionnalités sensibles (gestion des offres, gestion des candidatures, administration).

Cette configuration garantit que seules les requêtes autorisées provenant de l'application mobile ou du site web peuvent accéder aux ressources protégées.

Exemple de liaison

Les correspondances entre les écrans Android et les contrôleurs Symfony montrent clairement comment les deux parties de l'application communiquent pour assurer un fonctionnement cohérent entre mobile et serveur.

Correspondances entre activités Android et contrôleurs Symfony

- **LoginActivity (Android) ↔ SecurityController (Symfony)**
Cette association gère l'authentification. L'activité Android envoie les identifiants via l'API, tandis que le contrôleur Symfony vérifie l'utilisateur, applique les règles de sécurité et renvoie la réponse permettant ou non l'accès à l'application.
- **ListOffresActivity (Android) ↔ OffreController (Symfony)**
L'activité récupère la liste des offres de stage en interrogeant l'API. Le contrôleur Symfony fournit les données issues de la base, filtrées ou structurées selon les besoins, permettant l'affichage des offres sur mobile.
- **CandidatureActivity (Android) ↔ CandidatureController (Symfony)**
Cette paire assure la gestion des candidatures. L'activité Android affiche, crée ou modifie une candidature, tandis que le contrôleur Symfony traite les requêtes, applique les règles métier (états, validations) et met à jour la base de données.

Annexes table des matières

[Figure 1 : Architecture technique du projet Symfony](#)

[Figure 2 : Architecture de l'application mobile Android](#)

[Figure 3 : Diagramme de navigation et structure des composants](#)

lien

- Description : Organisation des dossiers du framework incluant le moteur de template (Twig), la logique métier (PHP) et la suite de tests (PHPUnit).

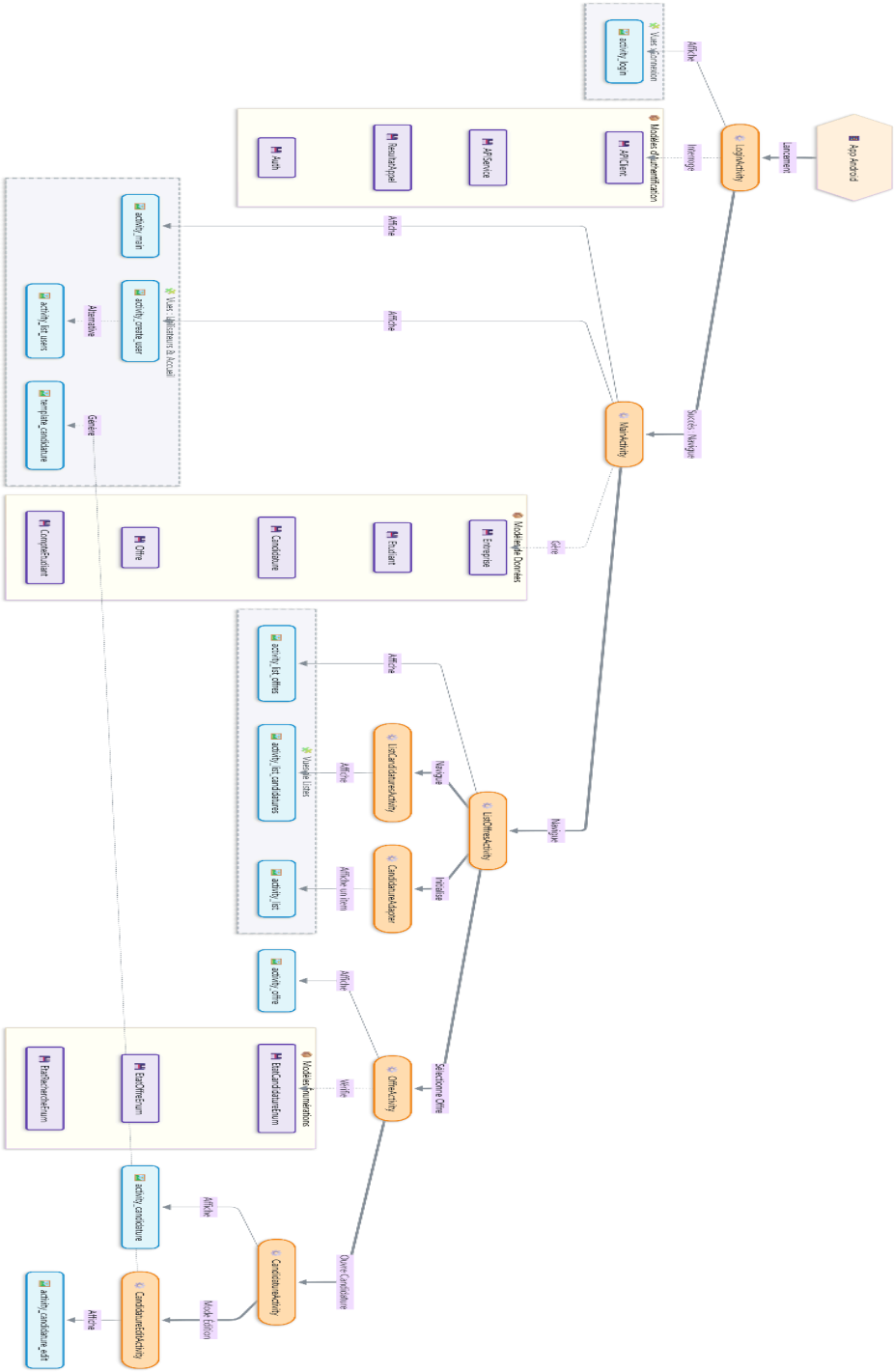


Figure 2 : Architecture de l'application mobile Android

- **Description** : Structure du projet Android Studio montrant la séparation entre l'interface (XML), la logique (Java) et la gestion de la compilation (Gradle).

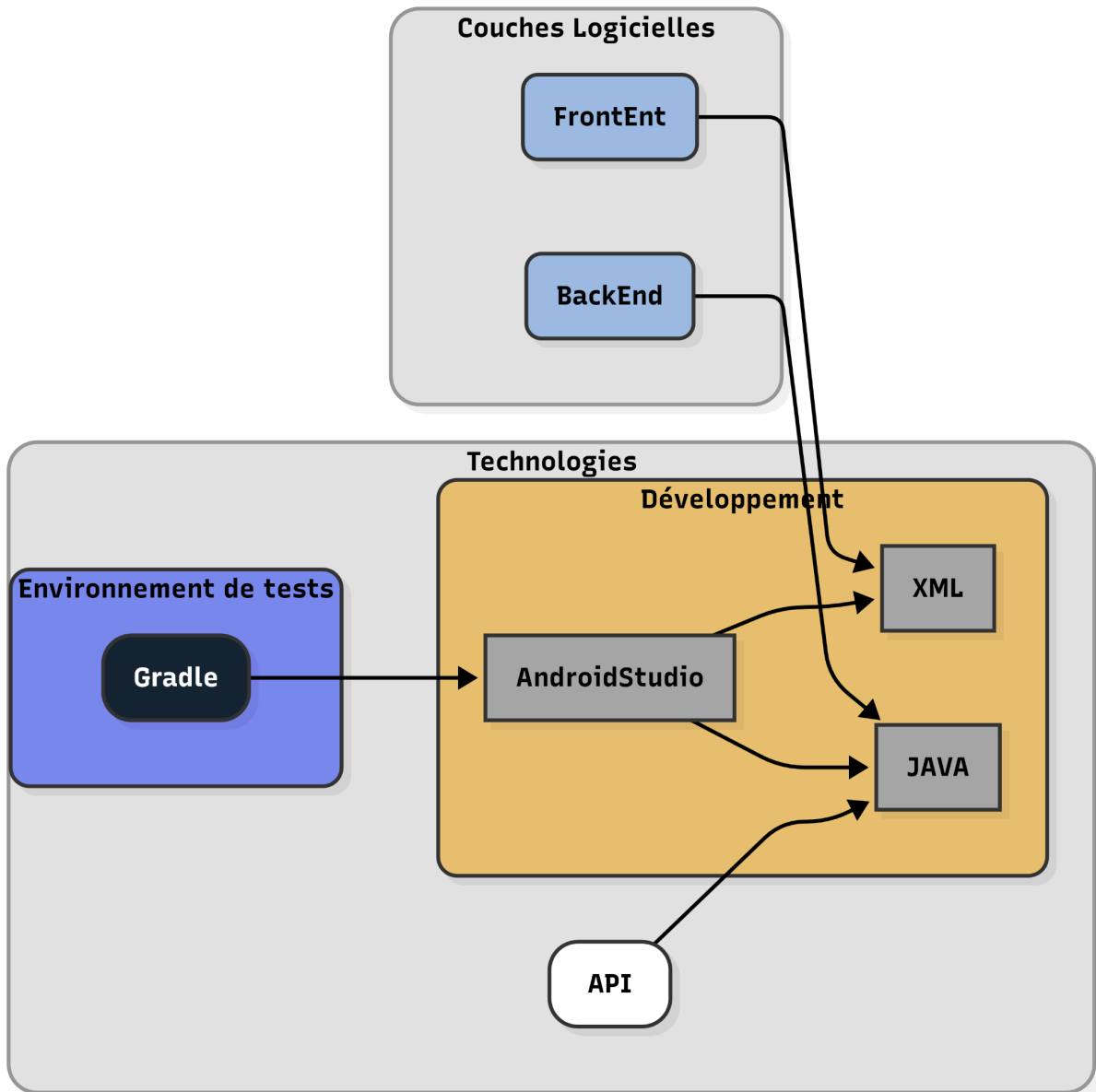


Figure 3 : Diagramme de navigation et structure des composants

- **Description** : Cartographie des interactions entre les vues (Fragments/Activities), les services de données et les modèles de l'application.

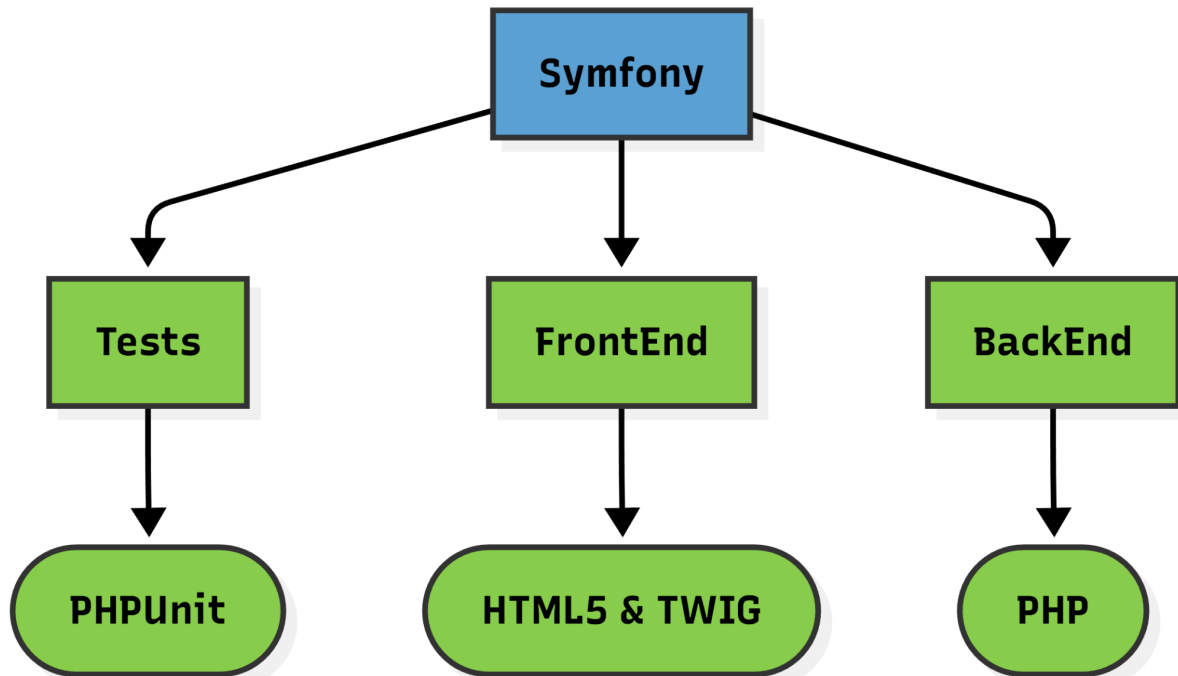


Figure 4 : Architecture globale Client-Serveur

- **Description** : Connexion via API REST entre l'application native Android et le backend Symfony 6. L'environnement serveur et la base de données SQL sont conteneurisés avec Docker.

